UNIMA — CENTRO UNIVERSITÁRIO DE MACEIÓ

PROJETO FINAL — ENGENHARIA DE SOFTWARE

FOCO
TAREFAS

Sistema de gerenciamento de tarefas pessoais

Planejamento · Requisitos · UML · Prototipação
Desenvolvimento · Testes · Documentação

Integrante único: Yuri Fernandes Bastos
Maceió — AL · 09/11/2026

1. Identificação

Projeto: Foco Tarefas. Disciplina: Engenharia de Software. Instituição: UNIMA — Centro Universitário de Maceió.

Integrante e responsável: Yuri Fernandes Bastos (único integrante).

Tipo: aplicação web local, responsiva e executável sem servidor. Prazo de entrega: 13/11/2026.

Materiais: código-fonte, README, quadro de gestão em GitHub, requisitos, dois diagramas UML, três protótipos, imagens do sistema e testes.

2. Introdução

Problema

Tarefas de estudo, trabalho e rotina frequentemente ficam dispersas em diferentes anotações. A falta de um único local dificulta identificar pendências, estabelecer prioridades e avaliar o progresso.

Justificativa e objetivo

O Foco Tarefas propõe uma solução de baixa complexidade que permite cadastrar, consultar, editar, concluir, excluir e organizar atividades. O objetivo acadêmico é aplicar de modo articulado as etapas de Engenharia de Software.

Público-alvo e escopo

Público-alvo: estudantes e pessoas que precisam gerenciar atividades individuais. Stakeholders: usuário final, responsável pelo projeto e professor avaliador.

Incluído: CRUD, categorias, prioridades, datas, busca, filtros, indicadores e backup JSON. Excluído: login, sincronização em nuvem, colaboração em tempo real e notificações.

3. Planejamento e gerenciamento

Modelo incremental

O desenvolvimento seguiu um modelo incremental: (1) problema e requisitos; (2) diagramas UML e protótipos; (3) implementação das funções essenciais; (4) testes e documentação. A abordagem foi escolhida por reduzir o risco de um sistema acadêmico pequeno.

Incremento	Atividades e entregas
1 - Planejamento	Definição do objetivo, stakeholders, requisitos e escopo.
2 - Modelagem	Casos de uso, classes e três telas prototipadas.
3 - Desenvolvimento	Interface HTML/CSS, regras JavaScript, CRUD e persistência local.
4 - Validação	Testes de lógica e interface, capturas, README e relatório.

Ferramenta de acompanhamento

O acompanhamento do projeto está registrado no GitHub Issues, com 13 itens numerados e atribuídos ao integrante Yuri Fernandes Bastos. O quadro Markdown publicado no repositório os distribui nas colunas A fazer, Em andamento, Em testes e Concluído, com links para as issues. O histórico de cada issue permite identificar estados e responsabilidades.

Quadro publicado: ver quadro de acompanhamento. Issues: ver atividades e histórico.

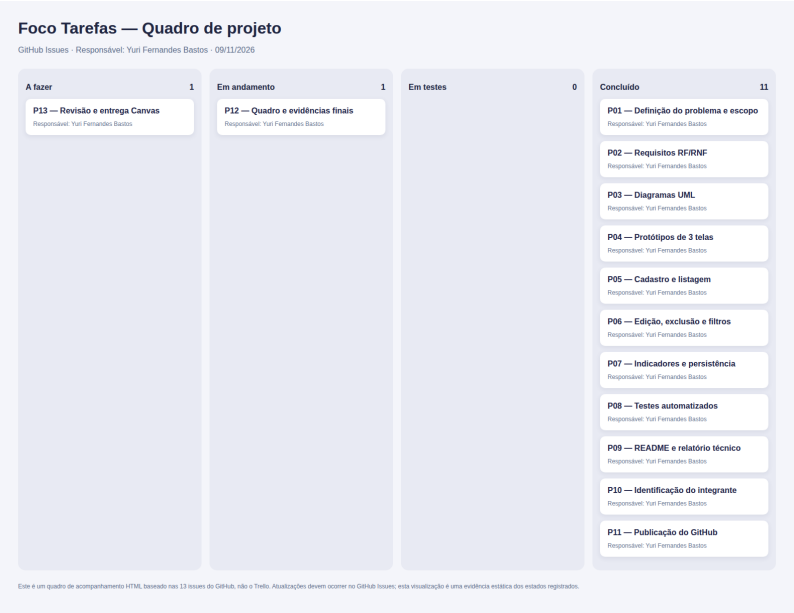


Figura 1 — Visualização HTML do acompanhamento registrado em GitHub Issues; não é um quadro Trello.

4. Engenharia de requisitos

Técnica de identificação de necessidades

Foi empregada análise do problema com observação estruturada de cenários simulados: criar tarefa, localizar uma pendência, alterar prioridade e marcar conclusão. Não foram realizadas entrevistas ou questionários reais.

Requisitos funcionais

Código	Descrição	Feito
RF01	Cadastrar tarefas com título obrigatório e demais campos.	Sim
RF02	Listar tarefas cadastradas.	Sim
RF03	Editar tarefa existente.	Sim
RF04	Excluir tarefa após confirmação.	Sim
RF05	Alterar status e marcar/desfazer conclusão.	Sim
RF06	Pesquisar e filtrar por status/categoria.	Sim
RF07	Exibir indicadores e percentual de progresso.	Sim
RF08	Importar e exportar arquivos de backup JSON.	Sim

Requisitos não funcionais

Código	Descrição	Feito
RNF01	Layout responsivo para desktop e celular.	Sim
RNF02	Execução local e persistência no navegador.	Sim
RNF03	Validação de campos e mensagens claras de erro.	Sim

5. Modelagem UML

A modelagem contém um diagrama de casos de uso (ator Usuário e ações sobre tarefas) e um diagrama de classes (Tarefa, TaskManager e dependência de armazenamento), ambos relacionados ao código.

Diagrama de casos de uso

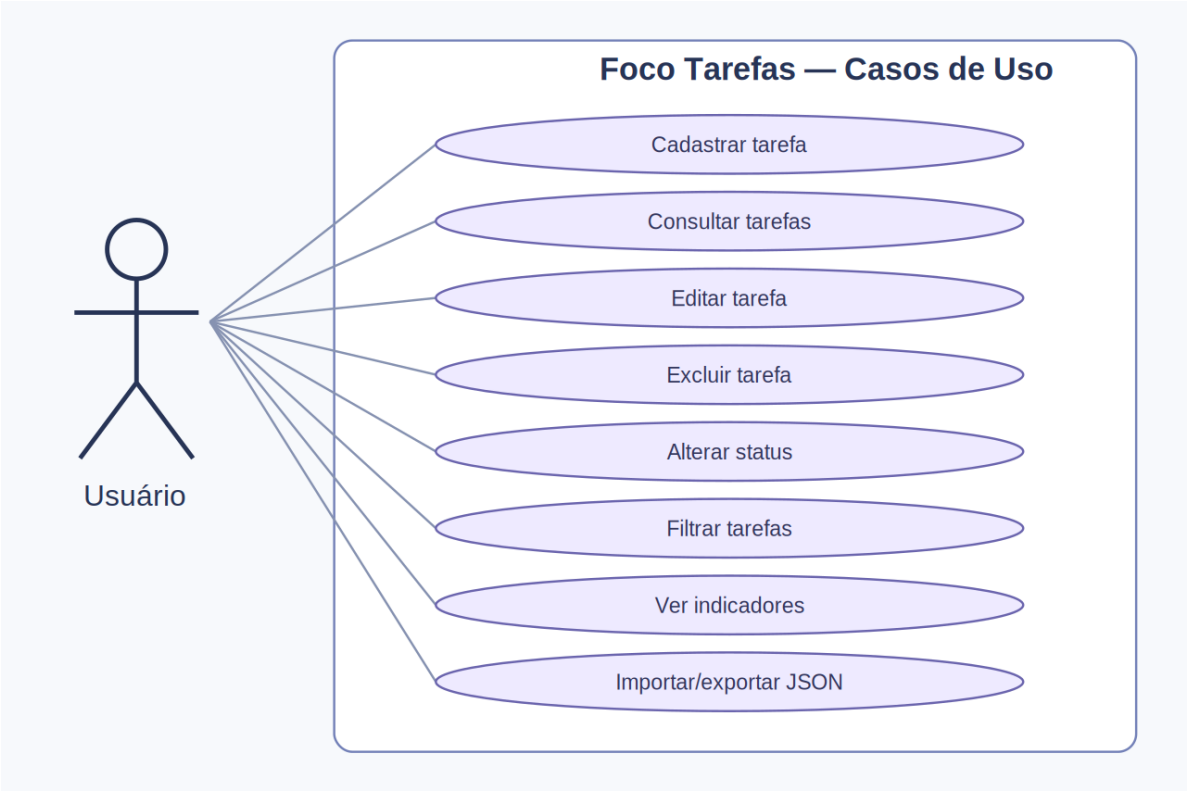


Figura 2 — Casos de uso do sistema.

Diagrama de classes

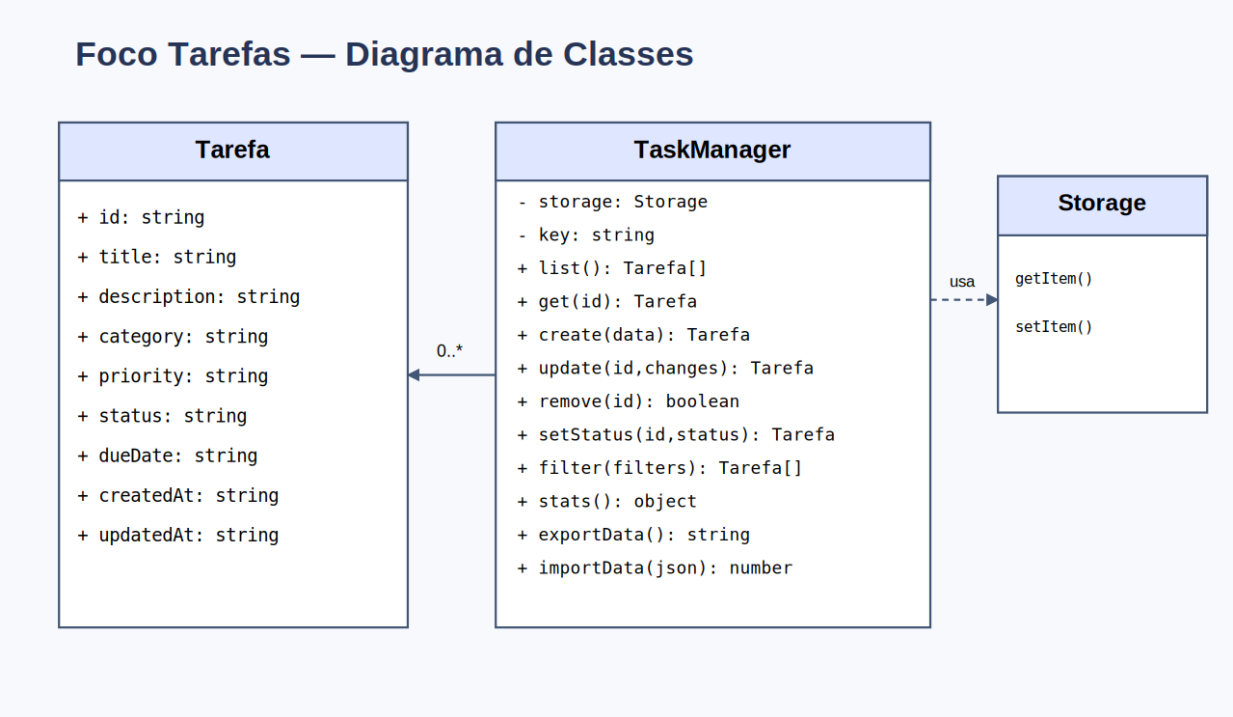


Figura 3 — Classes e dependências da implementação.

6. Prototipação

Foram preparados três protótipos de baixa fidelidade antes da versão final da interface, representando a navegação e a distribuição dos controles.

Tela 1 — Painel



Figura 4 — Protótipo do painel, vinculado aos requisitos RF02 e RF07.

Tela 2 — Listagem



Figura 5 — Protótipo da consulta e filtros, requisitos RF02 a RF06.

Tela 3 — Formulário

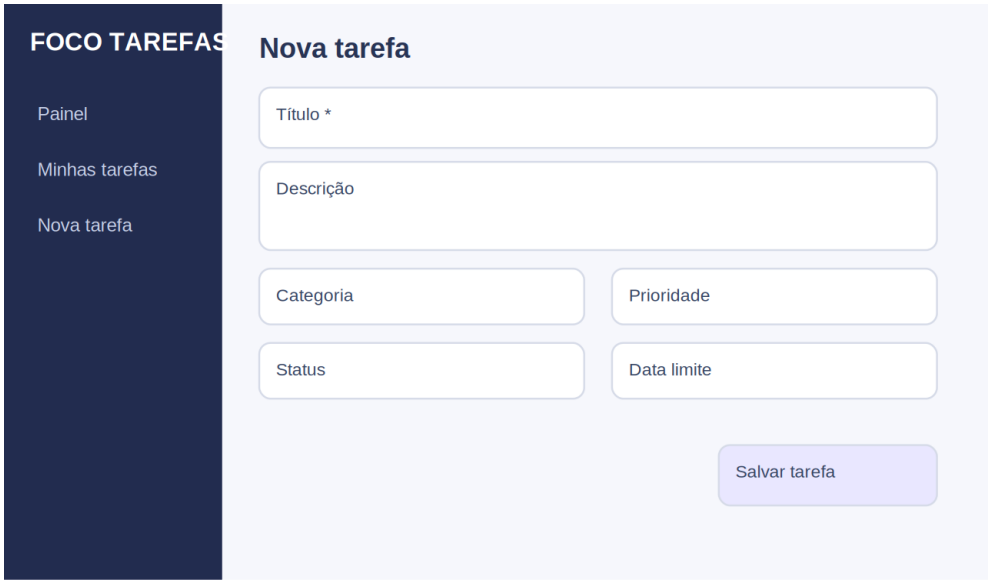


Figura 6 — Protótipo do cadastro e edição, requisitos RF01 e RF03.

7. Desenvolvimento do software

Arquitetura e tecnologias

O sistema utiliza HTML5, CSS3, JavaScript e localStorage. A classe TaskManager centraliza validação, persistência e CRUD; os arquivos CSS e JavaScript separados permitem manutenção, além de uma versão independente em arquivo HTML.

A aplicação não depende de instalação, servidor externo nem conexão de internet para uso normal. O repositório traz o código-fonte e o passo a passo de execução no README.

Funcionalidades implementadas e interface

As oito funções contemplam cadastro, consulta, edição, exclusão, mudança de status, pesquisa e filtros, estatísticas e backups JSON. As capturas abaixo foram produzidas com dados fictícios em navegador automatizado.

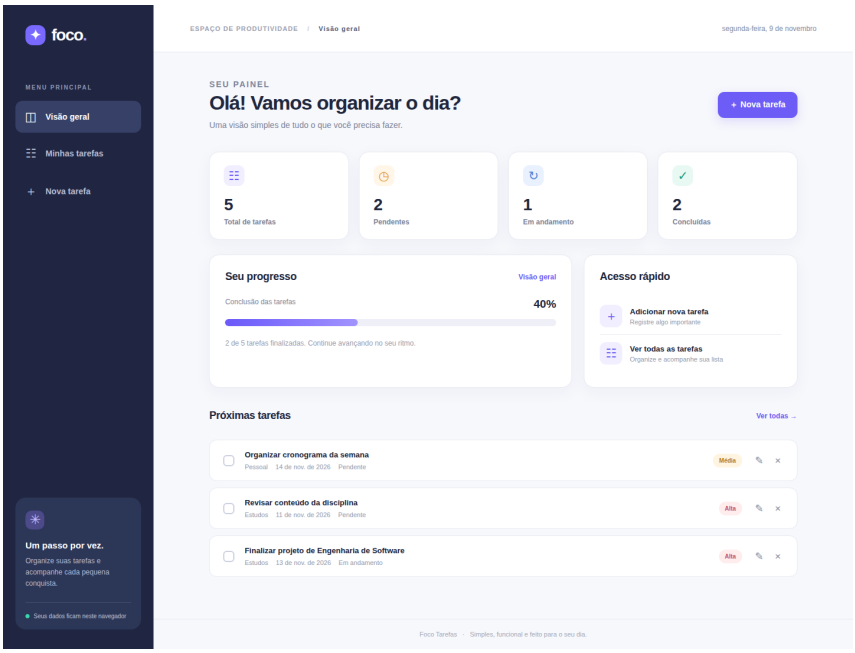


Figura 7 — Tela real do painel de indicadores.

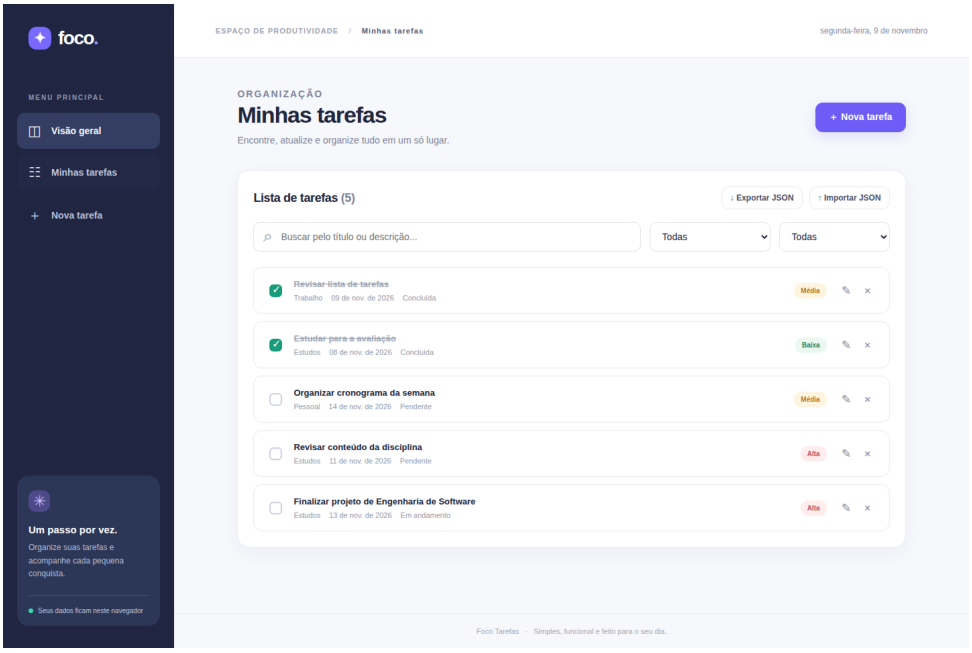


Figura 8 — Tela real de listagem e consulta.

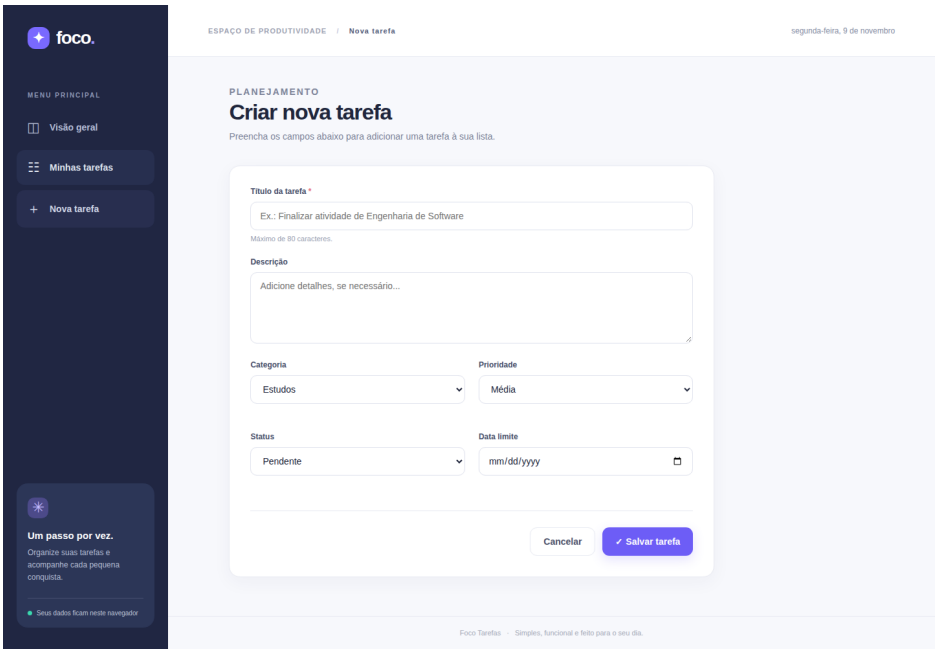


Figura 9 — Tela real do formulário de tarefa.

8. Testes e validação

Execução automatizada

A suíte Node.js (node:test) cobre dez cenários de regras de negócio; a automação visual Playwright/Chromium executa doze verificações de interface. Os registros são armazenados em docs/resultado-testes-node.txt e docs/evidencias-testes-ui.txt.

ID	Procedimento	Esperado	Obtido	Situação
CT01	Cadastrar tarefa válida	Incluir na lista	Registro incluído	Aprovado
CT02	Pesquisar e filtrar tarefas	Somente resultados	Filtro aplicado	Aprovado
CT03	Editar uma tarefa	Dados alterados	Dados alterados	Aprovado
CT04	Concluir uma de duas tarefas	50% de progresso	50% de progresso	Aprovado
CT05	Excluir registro existente	Registro removido	Registro removido	Aprovado
CT06	Cadastrar título vazio	Bloquear ação	Cadastro bloqueado	Aprovado
CT07	Exportar/importar JSON	Restaurar tarefas	Tarefa recuperada	Aprovado
CT08	Importar JSON inválido	Rejeitar arquivo	Arquivo rejeitado	Aprovado
CT09	Prioridade inválida	Rejeitar dados	Dados rejeitados	Aprovado
CT10	Reabrir gerenciador	Recuperar dados	Dados recuperados	Aprovado

As verificações UI01–UI12 também abrangem painel, listagem, busca, filtro, cadastro, persistência após recarregar a página, edição, conclusão, exclusão, exportação, erros de JavaScript e largura móvel.

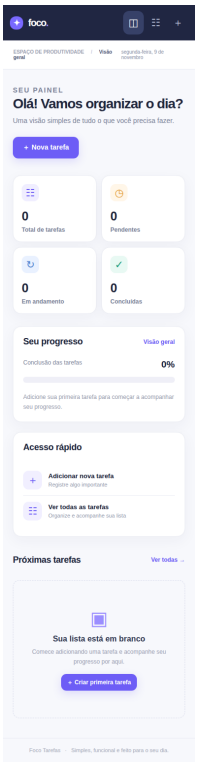


Figura 10 — Responsividade da interface em 390 pixels.

9. Considerações finais

O projeto demonstrou que a Engenharia de Software abrange análise de necessidades, planejamento, modelagem, protótipos, implementação, testes e documentação, além da programação. O produto final permanece simples e tem operações funcionais ligadas aos requisitos.

Dificuldades abordadas: validação de dados importados, ligação entre interface e armazenamento, estados das tarefas e consistência dos testes. Limitações: sem contas, nuvem ou colaboração. Melhorias futuras: login, alertas de prazos, sincronização e relatórios.

10. Repositório e entrega

Repositório GitHub: https://github.com/bastoss-dev/university-homework/tree/main/foco-tarefas.

Quadro de gerenciamento: https://github.com/bastoss-dev/university-homework/blob/main/foco-tarefas/docs/planejamento/quadro-online.md.

GitHub Issues: atividades atribuídas e histórico.

Responsável único: Yuri Fernandes Bastos.

Para executar: baixar o repositório e abrir index.html no navegador. Para executar testes de regra: node --test tests/core.test.js.

Entrega: até 13/11/2026 pelo Canvas, com este PDF e os links solicitados. A conferência final no computador do aluno e a submissão pelo Canvas não são realizadas pela automação.

A produção técnica e editorial contou com assistência automatizada. O estudante identificado deve conferir conteúdo, integridade e conformidade acadêmica antes de entregar.